Projects in Practice

From **Unknown Unknowns** to **Known Knowns** through **Known Unknowns**.

Identifying Unknown Unknowns

Projects start with **unknown unknowns**: we don't know what we don't know.

- Use tools and rules to reach **known unknowns** quickly.
- Reduce important unknowns early; plan for the rest.

Four States of Knowledge

1. **Unknown unknowns** — we don't know what we don't know *(most dangerous)*
2. **Known unknowns** — we know what to learn
3. **Known knowns** — we know it and can do it
4. **Unknown knowns** — we know it but don't realize it

Move: unknown unknowns → known unknowns → known knowns.

Iterative Process

- **Generally:** a sprint is a fixed 1–4-week period.
- **Here:** Sprint 1 and Sprint 2 are larger development iterations; each may span several such periods.

- **Sprint 1:** reveal unknowns quickly. Revise infeasible features/requirements based on prototypes or the MVP.
- **Sprint 2:** build out features using what we learned.

Prototypes

Prototype to test feasibility when the work is unfamiliar.

- **Throwaway:** answer the question; skip production-grade design and tests.
- **Potential product:** keep a basic data model and code quality.
- Expect throwaway code to be discarded later.

MVPs

MVP (Minimum Viable Product): a working product with minimum useful features.

- Build it quickly in Sprint 1; improve it later.
- *In this course, the MVP needs design, tests, and documentation.*

Definition of Done

Agree on what **Done** means:

- Reviewed code
- Passing tests
- Met acceptance criteria
- Updated documentation
- No critical known defects

Dogfooding, Milestones, Quality

- **Dogfooding:** use your own software to find usability problems and missing features.
- **Milestones** are checkpoints; plan backward from the **deadline**.
- Test and metric counts alone don't prove quality.

Aim for High-Quality

Build software that works and is easy to maintain:

- Easy to find and fix bugs
- Easy to add features

Quality Is More Than Maintainability

High-quality software is also:

- **Correct, usable, and reliable**
- **Secure**

Good architecture and design sustain problem-solving over time.

Professor (as Manager)

- **Manage People & Politics:** Navigate complexity and team dynamics.
- **Evaluate Teams:** Assess tech leads and team members.
- **Support Problem-Solving:** Provide guidance and resources.

Team Leaders (Tech Leads)

- **Manage Team Complexity:** Organize people and tasks.
- **Report Upward:** Keep managers informed of team progress.
- **Set Direction:** Define goals and schedules.
- **Drive Progress:** Ensure team members move forward.
- **Manage Artifacts:** Code, tests, and documents.

- **Lead Weekly Meetings:** Present team progress; delegate if absent.
- **Lead Ceremonies:** Guide PPP, S1R, S1P, S2R, and FP. S1R/S2R are internal retrospectives.

See Team Project slides for the acronyms.

Team Members

- **Tackle Complexity:** Design and implement solutions.
- **Model the Data:** Define data and its flow through modules.
- **Design Interfaces:** Define modules and interfaces that use the data.
- **Build & Test:** Write code and tests.

- **Raise Issues:** Report problems early.
- **Mentor Others:** Support newer teammates.

> Work independently, explain decisions, report issues early. AI automates routine junior tasks; solve problems like a senior engineer.

Four Professional Team Rules

1. **No Surprises** — communicate early
2. **Stay Constructive Under Pressure** — facts, not emotions
3. **Be a Professional** — deliver, respect, own mistakes
4. **Understand Others before Being Understood** — listen first

The next slides explain each rule.

No Surprises

> Unexpected problems break trust more than difficult problems.

1. **Communicate Early:** Report issues immediately.
2. **Be Transparent:** Share progress, blockers, and risks.
3. **Avoid Last-Minute Shocks:** Keep everyone informed.

Stay Constructive Under Pressure

> Stay respectful and fact-based, especially under pressure.

1. **Manage Emotions:** Respond with facts, not frustration.
2. **Focus on Solutions:** Discuss data and solutions, not blame.
3. **Build Trust:** Be steady and speak up about problems.

Be a Professional

> Professionalism is about actions and results, not titles.

1. **Deliver on Promises:** Do what you say, on time.
2. **Respect Others:** Be fair and courteous.
3. **Take Responsibility:** Own mistakes and learn from them.
4. **Strive for Quality:** Aim for excellence in every task.

Understand Others before Being Understood

> Listening deeply builds trust faster than speaking first.

1. **Seek Context:** Learn the problem's origin and who is involved.
2. **Listen Actively:** Hear and acknowledge concerns before offering input.
3. **Respond Thoughtfully:** Build solutions on shared understanding.

Start Early and Reduce Uncertainty

> We can plan well only once we've reduced the important unknowns.

1. **Start Early:** Reveal unknowns quickly.
2. **Prototype First:** Test feasibility and the problem fit before the MVP.
3. **Seek Clarity:** Clarify the problem as you work.

Deliver a Small, Working Result First

> A rough solution that works is more valuable than a perfect one that never ships.

1. **Start Simple:** Build a small end-to-end slice, not isolated layers.
2. **Prove Value:** Show it solves the right problem before refining it.
3. **Refine Later:** After proving value, improve speed and polish.

Improve Using Evidence and Feedback

> Good systems and habits make future work easier, not harder.

1. **Iterate:** Use feedback and results to improve.
2. **Reduce Friction:** Lower mental load with habits and tools.
3. **Stay Consistent:** Deliver reliably, whoever does the work.